HOB: A Free, Open-Source Obsidian-Based Research Management System for Scientific Workflows

Manuel Tiburtini[1]

[1]Fermo, Italy, ORCID: 0000-0003-4985-3058

Abstract

Scientific research requires the coordination of projects, literature, experimental records, writing, grants, conferences, administrative tasks, and indicators of research activity. Existing tools commonly address these functions in separate environments. This article presents HOB (HOB–Research Vault Toolkit), a free and open-source Obsidian vault template that provides an integrated research management system for researchers. HOB adds a structured operational layer to Obsidian through 11 stage-based folders, 18 templates for generating linked workspaces, a central dashboard, and nine scheduled macOS jobs. Its principal subsystems support Zotero-to-Markdown literature ingestion, conference documentation and local transcription, Bayesian project-duration forecasting, KPI collection, automation monitoring, and vault-aware AI search. The primary research records remain ordinary Markdown files that can be inspected, moved, linked, and versioned with standard tools. Optional language-model functions can use locally hosted Ollama-compatible endpoints, preserving a local-first workflow when configured accordingly. This article describes the architecture and implementation of the initial beta release and discusses its current limitations, including macOS-dependent scheduling, setup requirements, and the absence of dedicated multi-user collaboration features. HOB is distributed under the GNU Affero General Public License version 3 (AGPL-3.0) at `https://github.com/Bendlexane/HOB` and archived at `https://doi.org/10.5281/zenodo.22059429`.

Keywords: Research Management System; Obsidian; local-first software; Markdown; research workflow automation; Bayesian forecasting; open-source software

Version status. This preprint documents the beta release of HOB 1.0 available in August 2026. Both the software and this manuscript remain under active development and may be corrected or revised as features, configuration, documentation, and platform support evolve. The versioned software releases and repository history provide the authoritative record of subsequent changes.

1 Introduction

Scientific research is increasingly multidisciplinary, team-oriented, data-intensive, and collaborative. Although tools for data analysis and scholarly publishing continue to advance, the systems used to manage projects, grants, literature, experimental records, writing, administrative tasks, and performance metrics remain fragmented and often depend on separate services. Large-scale surveys of scholarly tool use reveal the fragmentation of research workflows, which are distributed across a wide range of specialized applications and activities [12]. In this context, reference managers such as Zotero [4] organize publications and annotations but do not manage complete project lifecycles. General-purpose project platforms track tasks but are rarely integrated with academic databases, local literature libraries, or research-specific records. Note-taking applications are flexible, but they usually require users to design their own organizational structure and automation.

Researchers must therefore preserve context across multiple applications and manually synchronize information among them. This fragmentation reduces productivity and increases administrative effort,

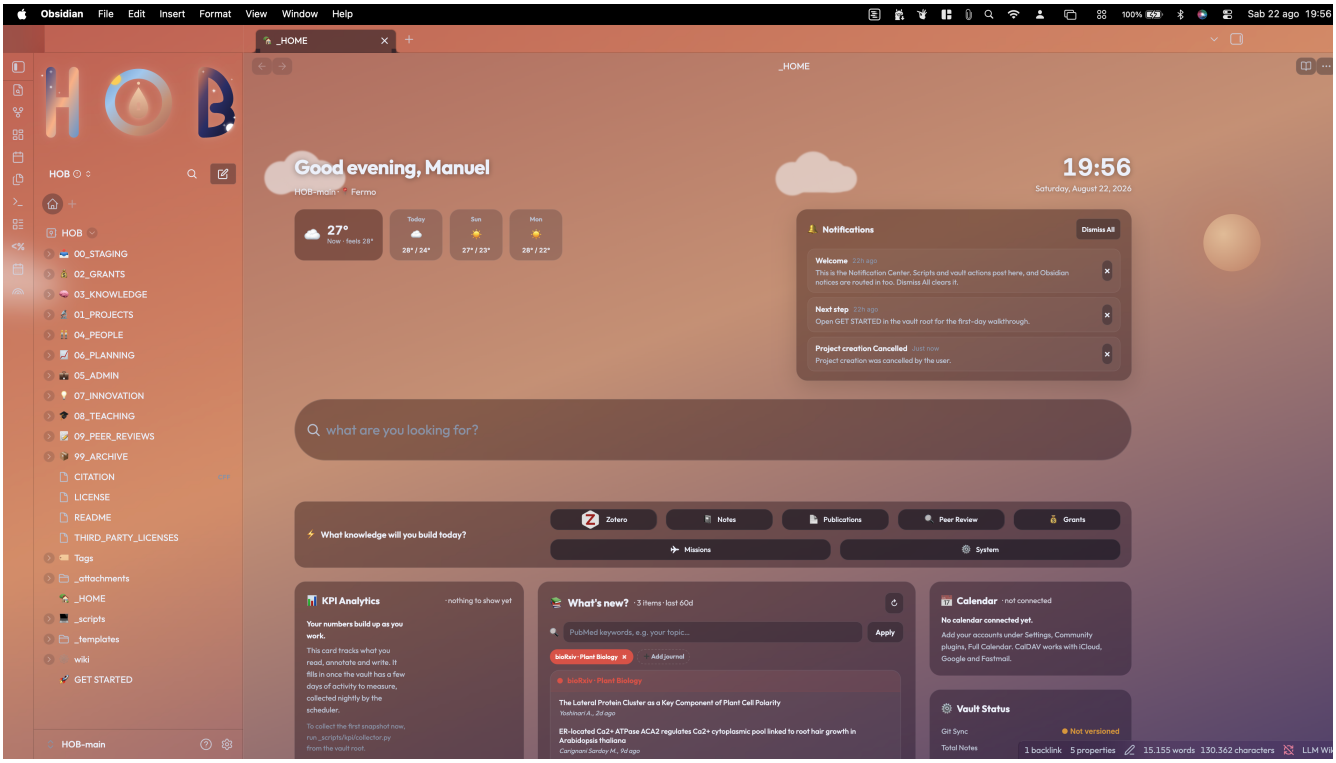


Figure 1: The Obsidian interface configured with the HOB toolkit, showing the central dashboard. It combines daily navigation, quick actions, weather and calendar information, research-activity indicators, automation status, literature updates, notifications, and vault-aware AI search within a single workspace. The colors shown correspond specifically to the evening appearance of the HOB Glass theme.

making it difficult to maintain a coherent record of decisions, evidence, deadlines, and outputs throughout a project's lifecycle.

Several software categories address individual parts of this problem. Electronic laboratory notebooks (ELNs) focus on experiments, protocols, inventories, and collaboration, improving the structure, searchability, and reuse of laboratory records [5]. Open-science platforms emphasize registration, preservation, and dissemination; OSF, for example, was developed as an open research-workflow environment linking projects and outputs [8]. General-purpose project-management systems organize tasks and deadlines, while reference managers organize literature. These categories overlap, but they usually treat research management as either laboratory record keeping, institutional data governance, open-output dissemination, or generic task coordination. Researchers and research teams may still need to connect these systems manually to manage grant preparation, literature triage, project planning, writing, administrative work, performance monitoring, and long-term knowledge retention.

AI-assisted tools add another layer to this landscape. Large language models can support activities such as information retrieval, literature synthesis, drafting, and coding [28]. However, when AI is provided as a separate web service, it may create an additional information silo.

This article presents HOB, a free, privacy-preserving, customizable, and open-source Obsidian vault toolkit designed to function as a *Research Management System* (RMS) [20, 19].

In this context, an RMS coordinates activities across the research lifecycle and extends beyond experimental documentation to projects, grants, literature, planning, writing, reporting, and workflow automation. HOB is distributed as a complete Obsidian workspace rather than a single plugin, and its name evokes the helpful household spirit of northern English folklore—a metaphor for software that quietly performs organizational work in the background [1]. Users can adapt it to their specific needs through Obsidian's themes and plugins. Its purpose is to provide an out-of-the-box Obsidian vault toolkit for

scientific work, not to replace analytical software, repositories, or institutional systems, but to connect them through a coherent operational layer.

Its main components are (i) a central dashboard (`_HOME.md`) for coordinating daily activities, with a clock, weather, calendar, system-status indicators, AI-powered search and explanations based on an indexed LLM Wiki, and quick-action buttons; (ii) a library of 18 structured Obsidian templates for projects, grant applications, missions, protocols, laboratory records, and other notes, giving each item a clear hierarchical structure with consistent folders, YAML frontmatter, sections, and internal links; (iii) a macOS `launchd`-based scheduler coordinating nine recurring jobs, including health checks, Bayesian posterior updates, KPI collection, wiki linting, and cleanup tasks that help HOB remain clean and functional; (iv) a Zotero pipeline that extracts annotations, classifies papers through an LLM, and generates Markdown literature notes; (v) a Bayesian Normal-Inverse-Gamma (NIG) model that predicts research-phase durations from the vault's Git history and adds a "today" line to Mermaid Gantt charts; (vi) local private databases that collect collaborator information, Git activity, Zotero reading metrics, researcher-oriented KPIs, and funding-runway data for dashboard visualization and spreadsheet export; (vii) a self-observability subsystem ("ops") that monitors automation health, validates note frontmatter against JSON Schema definitions written in YAML, and exposes failures in the dashboard's Notification Center; and (viii) an AI chatbot interface that queries an Ollama-compatible model for vault-aware assistance.

Together, these components place AI assistance within the same structured environment used for projects, literature, and planning, reducing the need to transfer context repeatedly between separate tools. Selecting a locally hosted model can also reduce the transmission of sensitive working material to external providers. HOB's knowledge layer draws on Andrej Karpathy's LLM Wiki pattern, in which an LLM incrementally maintains a persistent, interlinked Markdown knowledge base between raw sources and subsequent queries [9].

HOB follows three design principles. The first is *local-first* at the level of primary records: notes remain on the researcher's computer, the user retains direct access to and full control over the underlying files, and sharing remains at the user's discretion [11]. HOB is not local-only out of the box, because the default `gpt-oss:120b-cloud` model is hosted and requests sent to it leave the machine through the local Ollama daemon. However, users who require a fully local workflow and have the necessary computing resources can select a locally served model. The second principle is *stage-based organization*, so records are filed by workflow state rather than topic. The third is *progressive enhancement*, so the vault remains usable before optional scripts and AI services are enabled. By default, AI components can access only content stored in `03_KNOWLEDGE/`, which limits but does not eliminate the privacy implications of using a hosted model. The project is currently released as beta software under AGPL-3.0.

HOB's contribution is integrative rather than substitutive. It packages an editable research workflow in inspectable file formats and connects it to Zotero, Git, and Ollama. Obsidian allows users to modify content, metadata, plugins, themes, and CSS directly [17, 18], while HOB adds structured, vault-aware AI for automation, search, classification, synthesis, and question answering. The result is a free, extensible, ready-to-use, and citable research workspace rooted in an easy-to-use Markdown format.

2 Related Systems

HOB occupies a space between personal knowledge management, project coordination, ELNs, and open-research infrastructure. Table 1 compares HOB with five established systems using information published by their maintainers.

OSF is a free, open-source platform for creating and linking transparent research records. Its maintainers have announced that OSF Projects will stop accepting new projects on 16 November 2026 and become read-only on 19 February 2027, ending their role as a general collaboration, repository, and file-management workspace, while registrations, preprints, search, and the persistence of public links and DOIs remain unaffected [2, 3]. eLabFTW is an AGPLv3-licensed ELN that can be installed with Docker and self-hosted at no cost. Its maintainers also provide an optional paid hosting service, while the self-hosted

Table 1: Functional comparison of HOB and selected research-management platforms, based on official information reviewed in August 2026. Symbols indicate a standard capability (✓), a limited, tier-dependent, or partial capability (△), or a capability not advertised as standard (—).

Capability	HOB	OSF	eLab FTW	Sci Note	R Space	Lab Archives
No-cost entry	✓	✓	✓	△	△	△
Open or modifiable	✓	✓	✓	✓	✓	—
Local deployment	✓	△	✓	△	✓	—
Preconfigured workflow[1]	✓	✓	△	✓	✓	✓
Personal RMS scope	✓	△	△	△	△	△
Extensible	✓	✓	✓	✓	✓	△
AI support	✓	—	—	✓	—	△
Obsidian layer	✓	—	—	—	—	—
Citable or exportable	△	✓	△	△	✓	△
Personal KPI and forecasting	✓	—	—	—	—	—

[1] "Preconfigured workflow" denotes a system distributed with a populated domain structure of folders, templates, and automations; systems that require users to define their own templates before productive use are marked as partial. "Local deployment" denotes a maintainer-documented self-hosted deployment; open code without a supported self-hosting path is marked as partial. "AI support" includes both native AI features and supported external AI integrations, which differ in scope, hosting, and privacy. "Obsidian layer" refers specifically to customization through an Obsidian vault, community plugins, themes, CSS, and compatible scripts. "Cite/export" combines persistent-record, DOI, repository-publication, and structured-export pathways, which are not equivalent; systems offering structured export without persistent identifiers, or persistent identifiers only in higher subscription tiers, are marked as partial. The final row refers specifically to the combination of personal KPI collection and Bayesian project-duration forecasting described in this article, rather than to generic workflow automation.

edition retains the full set of software features. eLabFTW focuses on experiments, resources, templates, inventory, and scheduling, laboratory collaboration, interoperable export, and timestamping [6, 7]. SciNote combines an MPLv2-licensed open-source codebase with a free plan for individual users and paid hosted or local plans, emphasizing structured ELN workflows, integrations, APIs, data export, and AI-assisted protocol conversion and automation [24, 26, 23, 25]. RSpace provides an ELN and research-data-management platform that has been AGPLv3-licensed since June 2024. The supported Team and Enterprise editions require a paid subscription and add private or on-premises deployment, administrative controls, inventory management, additional integrations, and longer-term data retention [22, 21]. LabArchives provides a hosted ELN with a free edition limited to 1 GB of storage and paid Professional and Enterprise plans; inventory, scheduling, developer-API capabilities, DOI-based publication, and external AI image-integrity screening vary by product, function, integration, or subscription tier [13, 15, 14]. Unlike these systems, the HOB toolkit itself requires no user account because it is built around a local Obsidian vault.

2.1 Interpretation of the Functional Comparison

The matrix shows that HOB is not the first system to address research management and that several of its capabilities are already present in established platforms. Open-source licensing, local deployment, structured templates, integrations, data export, and AI-assisted functions are not unique in isolation. HOB's contribution lies instead in assembling these capabilities within a single researcher-oriented environment and making them available through a ready-to-adapt Obsidian vault.

The most distinctive aspect of this approach is its Markdown-first architecture. HOB is implemented directly within Obsidian, and its primary artefacts are ordinary files rather than records accessible only through a proprietary web application or an application-specific database. Researchers can therefore read, edit, search, link, version, and reorganize their material as needed using familiar tools while benefiting from Obsidian's ecosystem of plugins, themes, and CSS customization [17, 18]. The combination of Git history, activity indicators, health checks, and Bayesian duration estimates also makes aspects of the research process observable. In addition to storing results, HOB can help researchers understand how

their work is organized, where time and attention are being directed, and how previous projects can inform future planning.

The current beta is primarily a single-researcher system: it does not provide native real-time co-editing, user roles, access controls, or dedicated conflict-resolution mechanisms. Nevertheless, its consistent metadata, collaborator records, Git-compatible structure, and reusable templates were designed with future team workflows in mind. The framework can therefore be extended with new templates, automation scripts, integrations, visualizations, platform-specific schedulers, and collaborative functions to meet individual researchers' needs and support future team workflows. Because HOB is open-source and consists of inspectable Markdown, Python, JavaScript, and configuration files, users can adapt it to their disciplines and contribute improvements. If an active community develops around HOB, these contributions could broaden platform support, reduce setup effort, improve collaborative use, and allow the toolkit to grow from an individual research workflow into a shared research-management ecosystem.

3 System Architecture

The HOB vault is organized into five architectural strata that separate user interaction, workflow orchestration, domain services, persistent data with local runtime dependencies, and external network services (Figure 2). The separation is conceptual rather than physical: the vault holds the records, the templates, and the scripts, whereas the scheduler agents are installed under the user's `LaunchAgents` directory and the model server, the reference-manager database, and the document-conversion toolchain are independent local processes. Each stratum has a distinct responsibility and communicates with adjacent strata through commands, status information, and read–write operations.

Two properties of this arrangement are worth stating explicitly. First, direct network queries for literature feeds, calendar data, and weather originate in the presentation stratum, whereas every scheduled automation communicates only with processes on the local host. This describes the local socket, not necessarily the ultimate destination. When the configured Ollama endpoint serves a hosted model—including the default `gpt-oss:120b-cloud` configuration, which requires an Ollama account—the local daemon forwards requests to Ollama's remote infrastructure. Network confinement therefore holds strictly only when a locally served model is selected; AI-dependent jobs using the default hosted model require connectivity and may transmit supplied content beyond the local machine. Second, components do not call one another directly. Automations report through an append-only notification file that the dashboard renders, and in-application notices are routed into the same channel, which yields a single asynchronous bus between orchestration and presentation. User-triggered work follows the same principle of indirection through a declarative action registry that is the single source of truth for both the dashboard controls and the template menu, so actions are independent of the launcher that invokes them.

3.1 Vault Layout

HOB adopts an 11-folder, stage-based taxonomy (Table 2). Numeric prefixes preserve a stable order across platforms and in Obsidian's file explorer. Notes are placed in the directory corresponding to their *workflow stage*; for example, active projects are stored in `01_PROJECTS/` rather than in topic-specific folders. The People directory is reserved for records about students, collaborators, and coauthors rather than scientific content.

3.2 Dashboard

As shown in Figure 1, the dashboard (`_HOME.md`) uses a single `dataviewjs` block to render the daily entry point. Its upper section combines a greeting bar, a real-time clock, and weather information for a configurable location. A shared search field supports both conventional vault navigation and natural-language queries. Pressing Enter on a question routes the request to the Ask-AI panel through the

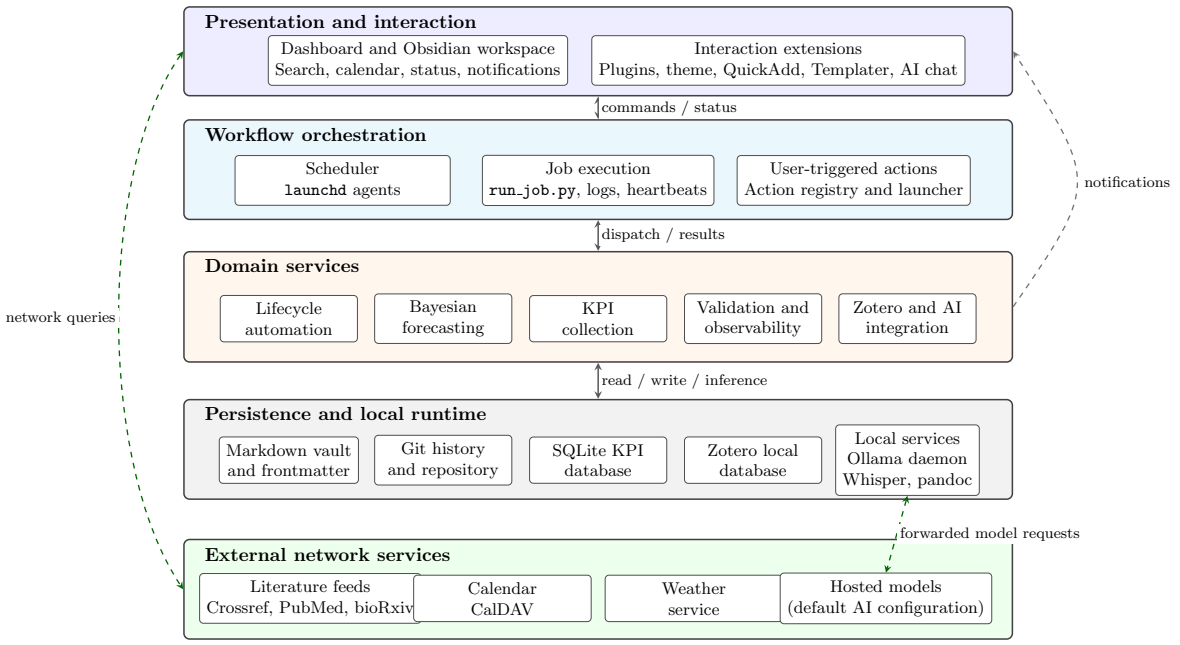


Figure 2: Five-stratum architecture of HOB. Solid connectors represent interfaces and data exchange between adjacent strata rather than a sequential process. Dashed connectors show direct presentation queries, model requests that may be forwarded by the local Ollama daemon to a hosted service, and asynchronous notifications from domain services.

`llm-wiki` plugin, which searches the indexed knowledge layer in `03_KNOWLEDGE/`. In AI mode, the answer is displayed directly beneath the search field together with the response time, a dismissal control, and an expandable list of the indexed sources used to construct the response (Figure 3).

Operational activities are exposed through the "Vault Actions" grid, which provides one-click creation of projects, grant applications, laboratory notes, peer reviews, and transcriptions using the corresponding structured templates. The dashboard also summarizes research activity through a KPI Analytics card containing sparklines for reading, writing, and the topic distribution of items processed in Zotero. A *"What's new?"* panel uses Crossref to retrieve recently published papers from journals identified by user-configured ISSNs, while a calendar panel displays events obtained from CalDAV sources through the Full Calendar plugin.

System state is consolidated in the same interface. The Vault Status card reports Git synchronization, note counts, enabled plugins, and the recency of the LLM index of `03_KNOWLEDGE/`. A vault-aware chatbot connects to an Ollama-compatible endpoint, and the Notification Center aggregates alerts generated by scheduled automations. The dashboard remains directly editable through Markdown, JavaScript, and CSS. Its visual layer uses the custom "HOB Glass" frosted-glass theme together with the Sky Background plugin, which renders a time-dependent sky behind the interface panels.

3.3 Scheduler

HOB uses macOS `launchd` LaunchAgents rather than bundling a separate scheduling service. Each agent runs in the user's session, while the `run_job.py` wrapper sets the working directory, captures logs, and records a heartbeat after successful execution. The current scheduler configuration is therefore macOS-specific, although most target scripts are portable Python programs.

Nine jobs are registered via `setup_launchd.py` (Table 3). The morning sequence is ordered by dependency rather than by convenience: posteriors are rebuilt before the Gantt charts that consume them.

The execution chain begins when `launchd` loads an agent property-list file. The `run_job.py` wrapper

Table 2: Top-level vault folder taxonomy.

Folder	Purpose
`00_STAGING/`	Inbox for daily notes, downloads, and raw ideas
`01_PROJECTS/`	One folder per active research project
`02_GRANTS/`	Grant applications (writing, submitted, active, archive)
`03_KNOWLEDGE/`	Literature notes, concepts, methods, and protocols; this directory forms the knowledge layer indexed by LLM Wiki
`04_PEOPLE/`	Students, collaborators, coauthor registry
`05_ADMIN/`	Bureaucracy, missions, certificates, CV
`06_PLANNING/`	KPIs, monthly plans, annual reports on research performance
`07_INNOVATION/`	System changes, new technologies, retrospectives, tech radar
`08_TEACHING/`	Courses, seminars, teaching materials
`09_PEER_REVIEWS/`	Reviews in progress for journals
`99_ARCHIVE/`	Finished projects (auto-moved by HOB after publication)

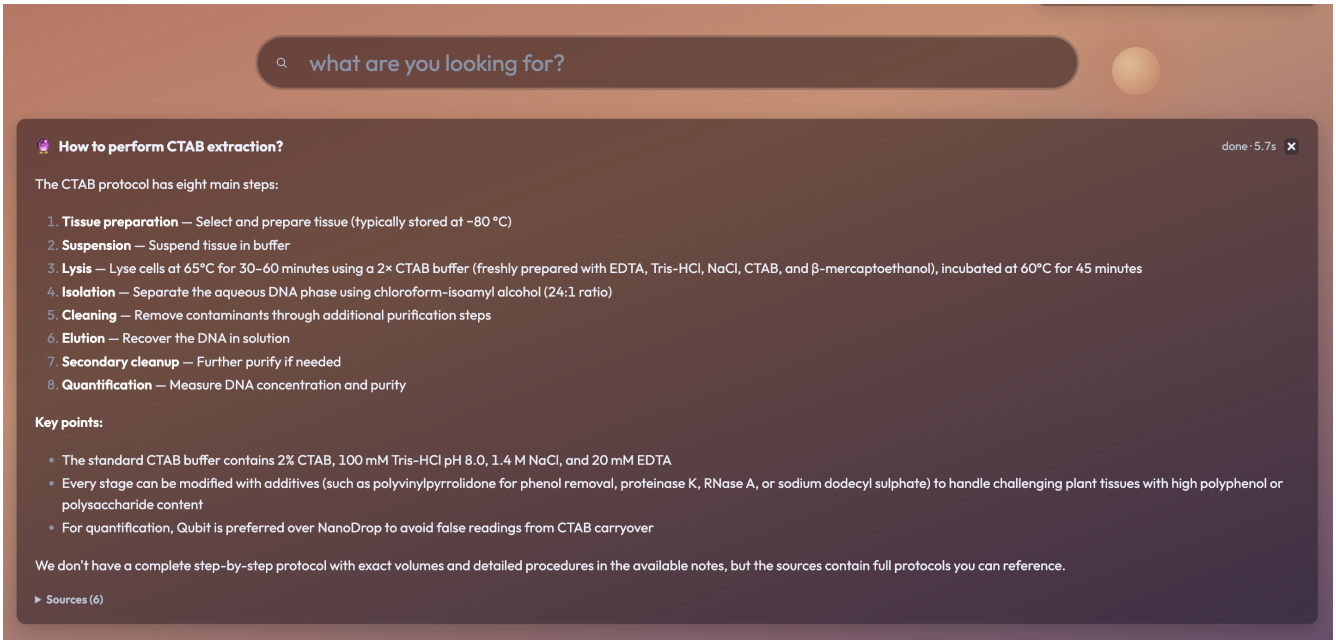


Figure 3: HOB dashboard in AI mode within the Obsidian interface. In this example, Ask-AI answers a question about CTAB extraction using the indexed `03_KNOWLEDGE/` layer and provides an expandable list of six supporting sources. The displayed response is generated from the contents available in the user's vault.

then sets the working directory, executes the target script, and captures its output to a job log inside the vault. On a clean exit it records a timestamped heartbeat; jobs that report to the interface additionally write into the notification file, so that overdue reviews or decaying ideas surface on the dashboard rather than in a log.

Failures are not handled synchronously. Detection is delegated to a separate liveness and health check, which distinguishes a job that never fired, using the heartbeat file or, in its absence, the modification time of the job log, from a job that fired but left a recognized error signature in its log tail. This is what allows a silently failing automation to be surfaced rather than assumed healthy, and it is also why the health check is the one agent registered without a heartbeat of its own.

Table 3: Scheduled automations registered by `setup_launchd.py`.

Agent	Function	Time
`health_check`	Runs every registered check and writes a dated health snapshot	07:55 daily
`archive_published`	Moves projects whose status is *published* into the archive	08:00 daily
`update_posteriors`	Rebuilds phase-duration posteriors from the priors and the git history	08:05 daily
`refresh_gantt`	Re-renders the Gantt charts of active projects from the updated posteriors	08:10 daily
`check_review_deadlines`	Flags overdue peer-review rounds and notifies the dashboard	09:00 daily
`kpi_collector`	Runs the KPI submodules and populates the metrics database	17:00 daily
`wiki_lint`	Removes polluted pages from the generated wiki layer and notifies	17:00 daily
`check_idea_decay`	Proposes promotion of project-linked ideas older than 30 days	17:00 daily
`work_life_balance`	Signals one hour ahead that the working day is ending	Mon–Fri, 17:00

3.4 Zotero Pipeline

HOB uses Zotero as its default reference library. The integration combines Python scripts stored in `_scripts/kpi/` with JavaScript actions executed through the Zotero Actions and Tags plugin. When a new item is created, `zotero_actions_tags_script.js` uses the configured Ollama-compatible endpoint to assign tags from the researcher's own taxonomy to the library item. During the nightly collection cycle, `collector.py` invokes `zotero_stats.py`, which reads the local `zotero.sqlite` database and automatically derives daily additions, reading activity, and tag distributions for the KPI system.

Annotation export is initiated manually within Zotero through `zotero_actions_tags_export.js`. The action resolves the bibliographic item associated with the open attachment and collects its title, authors, year, journal or book title, DOI, abstract, volume, issue, pages, tags, and Better BibTeX citation key. It then reads annotations from attached PDF files, preserving their text, comments, colors, page labels, types, and document order while excluding image-area annotations. The action processes PDF attachments for full-text operations; other Zotero attachment formats are not used as full-text sources by this workflow.

The user chooses among three processing modes that reflect the document type and desired level of analysis: (i) a standard mode that exports highlights and comments only; (ii) full-text synthesis, intended for journal articles and other papers; and (iii) chapter-by-chapter synthesis, intended for books or similarly structured long documents. The two synthesis modes require Zotero to resolve a local PDF path; if no PDF is available, the action automatically falls back to the highlights-only mode. The user can also select the destination folder and optionally associate the note with an existing HOB project. Available destinations include `00_STAGING/_currently-reading/`, `00_STAGING/`, and `03_KNOWLEDGE/literature/`.

After these selections, the JavaScript action writes the metadata and annotations to a temporary JSON file and asynchronously launches `zotero_export_background.py`. The Python worker receives the JSON file, PDF path, destination, processing mode, and project code; it performs the requested synthesis and creates a Markdown note named using the citation key. Notes exported to `03_KNOWLEDGE/literature/` are subsequently incorporated into the semantic index by `llm-wiki`, making the imported literature available to the dashboard's Ask-AI interface.

The integration targets an Ollama-compatible endpoint. The default configuration uses the hosted `gpt-oss:120b-cloud` model: although the script connects to an Ollama daemon through a localhost socket, that daemon forwards model requests and their supplied content to Ollama's remote infrastructure. When the endpoint instead serves a locally hosted model, prompts, annotations, extracted PDF text, and vault content remain on the user's machine. The two JavaScript actions must be installed and configured manually in Zotero before automated tagging and annotation export become available.

3.5 Conference Workspaces

Attending a conference produces two distinct kinds of record. The first is an administrative trail required for authorization and reimbursement, while the second documents the scientific content presented and discussed during the event. HOB represents both within a single generated workspace so that receipts, schedules, presentations, notes, recordings, and transcripts remain connected to the same event rather than being reconstructed afterwards from separate sources.

The conference action requests the event name, dates, location, contribution type, authorization status, estimated cost, and an optional link to an existing project. It then generates a workspace under `05_ADMIN/missions/conferences/`. Its folder structure separates administration, expenses, awards, and received materials. Expenses are further organized into receipts, invoices, accommodation, transport, and a free-form travel log. A folder for slides and posters is created only when the conference is not linked to an existing project, because project-related contributions remain with their project rather than being duplicated in the administrative workspace.

The scientific record begins with the conference programme. When one is supplied, HOB normalizes it, stores it as a typed note, and uses the configured language-model endpoint to generate one note for each conference day. Each daily note contains a Mermaid schedule and a session breakdown with times, speakers, and space for written observations. Re-running the extraction regenerates these notes from an updated programme, allowing the workspace to accommodate schedule revisions. Each talk includes a recording control. Its first activation identifies the session and begins audio capture, while the second stops the recording, transcribes it, and appends the transcript beneath the same heading used for the researcher's notes. Written and spoken records of a session therefore remain adjacent.

Transcription is performed by a local service bound to the loopback interface and started on demand by the recording control. The service requires a locally installed OpenAI Whisper model, which automatically detects the language spoken in each recording and produces the initial transcript. The default hosted `gpt-oss:120b-cloud` model then cleans and structures the text before it is appended to the corresponding session note. Requests include a shared secret generated during initial configuration to restrict access to the capture endpoint. Audio files and the initial Whisper outputs are retained within the conference workspace, but the default transcript-cleaning step sends the transcript through the local Ollama daemon to the hosted model. Programme parsing and transcript cleaning remain entirely local only when the user replaces the default with a locally served language-model endpoint.

3.6 Bayesian Research Project Forecasting

HOB implements a conjugate Bayesian Normal-Inverse-Gamma (NIG) model to estimate the duration of research-paper phases, such as data collection, analysis, and writing, from historical transitions reconstructed from the vault's Git log. The model produces estimated phase durations for each research project. The general strategy follows established work on Bayesian project-duration forecasting, in which prior schedule information is updated as observed performance becomes available [10]. Unlike the beta-S-curve model of Kim and Reinschmidt, HOB models log-durations directly and updates a NIG prior.

3.6.1 Model Formulation

Let D_i denote the duration of task i in calendar days. The model represents the log-duration as

$$Y_i = \log D_i \sim \mathcal{N}(\mu, \sigma^2) \tag{1}$$

with the following conjugate NIG prior.

$$\sigma^2 \sim \text{Inv-Gamma}(a_0, b_0) \tag{2}$$

$$\mu \mid \sigma^2 \sim \mathcal{N}\left(\mu_0,\ \frac{\sigma^2}{\kappa_0}\right) \tag{3}$$

Prior parameters are initialized from hand-elicited Beta-PERT three-point estimates (O, M, P) stored in `priors.json`, where O and P are elicited as approximate p_{10} and p_{90} durations and M is the mode. The Beta-PERT mean and variance are matched to a log-Normal by the method of moments, and the resulting log-scale parameters seed the NIG prior:

$$\begin{aligned} m &= \frac{O + 4M + P}{6}, & v &= \left(\frac{P - O}{6}\right)^2, \\ \sigma_0^2 &= \log\!\left(1 + \frac{v}{m^2}\right), & \mu_0 &= \log m - \tfrac{1}{2}\sigma_0^2, \\ \kappa_0 &= 0.5, \quad a_0 = 3, & b_0 &= \sigma_0^2\,(a_0 - 1). \end{aligned} \tag{4}$$

The location μ_0 is the log-scale mean implied by the moment match rather than the logarithm of the mode, and b_0 is chosen so that the prior mean of the variance, $b_0/(a_0 - 1)$, equals the elicited σ_0^2.

3.6.2 Conjugate Update

Given observed log-durations $y_1, \ldots, y_n$, define $\bar{y} = n^{-1} \sum_i y_i$. The posterior parameters are

$$\kappa_n = \kappa_0 + n, \qquad \mu_n = \frac{\kappa_0 \mu_0 + n\bar{y}}{\kappa_n}, \tag{5}$$

$$a_n = a_0 + \frac{n}{2}, \qquad b_n = b_0 + \frac{1}{2}\sum_{i=1}^{n}(y_i - \bar{y})^2 + \frac{\kappa_0 n(\bar{y} - \mu_0)^2}{2\kappa_n}. \tag{6}$$

The resulting posterior retains the conjugate form

$$\sigma^2 \mid \mathbf{y} \sim \text{InvGamma}(a_n, b_n), \tag{7}$$

$$\mu \mid \sigma^2, \mathbf{y} \sim \mathcal{N}\!\left(\mu_n, \frac{\sigma^2}{\kappa_n}\right). \tag{8}$$

3.6.3 Posterior Predictive

The predictive distribution for the log-duration of a new task is

$$\widetilde{Y} \mid \mathbf{y} \sim t_{2a_n}\!\left(\mu_n, \frac{b_n(\kappa_n + 1)}{a_n \kappa_n}\right), \tag{9}$$

a location-scale Student-t distribution with $2a_n$ degrees of freedom, where the second argument is the predictive scale squared. Transforming its quantiles back with the exponential function yields duration estimates p_{10}, p_{50}, and p_{90}. HOB uses these values to update Mermaid Gantt charts and display forecast uncertainty. The forecasting pipeline is implemented by `extract_history.py`, `update_posteriors.py`, and `refresh_gantt.py`.

3.7 KPI Pipeline

The KPI subsystem collects, stores, and exposes workflow indicators in a local SQLite database (`research_kpis.db`), whose schema separates daily aggregates from reading depth, topic breakdown, activity time, work pattern, and funding state. The collector (`collector.py`) runs daily at 17:00 and coordinates two acquisition modules, then persists and exports their output. It also accepts an explicit date, so that missed days can be backfilled, and a dry-run mode that reports what would be written.

Git statistics, computed by `git_stats.py`, derive daily word counts from Markdown diffs, counts of notes created or modified across Markdown, LaTeX, and Quarto files, and the commit count. Zotero

statistics, extracted by `zotero_stats.py` from an immutable read of the local Zotero database, measure additions, annotation depth, and papers revisited, and classify each item into a single reading category. Classification is performed by the configured language model using a prompt held in the vault. An explicit fallback category is used when the model is uncertain, making the topic breakdown reproducible rather than dependent on manual tagging.

Results are written to the daily, reading-depth, and topic tables, and then exported in two forms: a JSON snapshot consumed by the dashboard (`export_json.py`), and, on demand, an Excel workbook produced with `pandas` and `openpyxl` (`database.py`). Two further modules, funding runway from active grant records and citation metrics, are implemented but not yet wired into the daily collection, so their tables remain empty in the current release. All KPI data are stored locally.

3.8 Self-Observability (Ops)

The ops subsystem converts silent automation failures into visible, actionable alerts. It runs daily at 07:55, before the morning dashboard view, and performs three groups of checks. These cover (i) liveness, which verifies that each job declared in a registry of expected schedules ran within its interval, using heartbeat timestamps or, in their absence, log modification times; (ii) health, which searches the latest execution log for tracebacks, permission errors, and explicit failure markers, since a job may fire and still fail; and (iii) frontmatter integrity, which uses `Draft7Validator` to validate typed notes against JSON Schema definitions written in YAML and held in `schemas/`, with notes of untyped kinds reported as skipped rather than as passing.

Results are written as a JSON snapshot in `logs/YYYY-MM-DD_health.json`, retained for 90 days, and rendered as a Markdown block on the dashboard. The `--strict` flag exits with code 1 on any red status, enabling integration with external monitoring.

3.9 Plugins and Theme

HOB integrates eight Obsidian plugin components, which are declared in the vault's community-plugin configuration and distributed as prebuilt components inside the repository. Six are used without modification, while two are maintained specifically for HOB: the HOB fork of `home-tab` and a modified build of `llm-wiki`. The latter derives from Dominique Leca's Obsidian implementation of the LLM Wiki concept originally proposed by Andrej Karpathy [9, 16].

Table 4: Obsidian plugin components integrated into HOB.

Component	Role within HOB
Dataview	Query engine supporting the dashboard and KPI cards.
Templater	Template engine for the 18 structured templates in `_scripts/templates/`.
QuickAdd	Macro runner used by the Vault Actions quick-launch grid.
Full Calendar	Integration of CalDAV calendar sources.
Sky Background	Time-dependent sky rendering behind the interface panels.
Make.md	Folder notes, stickers, and the Spaces navigator.
llm-wiki *HOB fork*	Semantic indexing of the knowledge layer and inline Ask-AI search through an Ollama-compatible endpoint.
home-tab *HOB fork*	Folder search and routing of natural-language queries to Ask-AI.

Karpathy's original LLM Wiki proposal describes a persistent and compounding knowledge artifact maintained by an LLM rather than a conventional retrieval system that reconstructs an answer directly from raw documents for every query [9]. Leca packaged this general pattern as an Obsidian plugin [16]. The HOB fork of `llm-wiki` changes that plugin from a whole-vault indexer into a scoped component

of the research workspace. It adds folder-level inclusion and exclusion, permits index and metadata notes to be skipped, and is configured by HOB to process only `03_KNOWLEDGE/`. This restriction is relevant to privacy as well as performance because peer reviews, collaborator records, administrative documents, and unpublished project material should not enter the semantic index by default. The fork also exposes an inline query interface for the dashboard and divides long documents into overlapping chunks to preserve context across chunk boundaries. The modified build is released separately at `https://github.com/Bendlexane/llm-wiki`, branching from upstream commit `c99dae5`, so its differences from Leca's implementation remain publicly inspectable.

The visual identity of HOB is provided by "HOB Glass," a free and customizable theme distributed with the vault. It combines semi-transparent interface surfaces with the dynamic background generated by Sky Background and automatically adapts its contrast to daytime, nighttime, and weather conditions; this feature can be disabled. Its CSS source remains directly accessible for further modification. Standard Obsidian themes can be used as well.

3.10 Git Integration and Data Privacy

HOB uses Git to version the vault and reconstruct changes in project status over time; this integration must be configured by the researcher. Functions that depend on historical activity, such as data recovery or Bayesian duration forecasting, therefore require the vault to be initialized as a Git repository and committed regularly. A remote hosting service is not required for local operation, although researchers may configure one for backup, synchronization, or collaboration.

Remote synchronization must be configured carefully according to the sensitivity of the research material. Because the vault may contain private notes, unpublished results, participant information, credentials, environment files, local databases, recordings, and transcripts, pushing the complete vault to a public repository or public fork can unintentionally disclose confidential research data. **Researchers must therefore verify that their own remote repository is appropriately protected, configure `.gitignore` before the first commit, and review all staged files before every push.** Deleting an exposed file from the latest revision may be insufficient because copies can remain in Git history, caches, clones, or forks. **Responsibility for preventing such disclosure and for ensuring compliance with applicable ethical, institutional, and data-protection requirements remains with each researcher operating their own private fork of the vault.**

4 Discussion and Conclusion

HOB originated from the need for a general, scalable, and accessible Markdown-based Obsidian environment in which projects, literature, administration, research activities, and other aspects of a researcher's daily work could be managed without fragmentation across unrelated services or reliance on desktop folders alone. Its stage-based structure organizes related records into connected workspaces, while its Markdown-first design allows information to be moved, linked, searched, versioned, and reused with ordinary tools and common AI tools. At the same time, the volume of scientific information is expanding at a pace that increasingly exceeds researchers' capacity for manual review; AI-assisted retrieval, classification, and synthesis are therefore becoming indispensable tools for keeping pace with the literature and maintaining usable research context [28]. Obsidian provides the underlying capabilities for local file access, properties, internal links, search, graph navigation, plugins, and interface customization [20, 19, 17]; HOB does not replace these functions, but adds a research-oriented framework comprising a predefined taxonomy, structured templates, integrated services, and automated workflows. This approach preserves researcher control because core records remain in a user-managed directory, can be synchronized through Git or another file-sharing mechanism, and can be processed locally, including through optional locally hosted AI models. Bayesian forecasts and KPI summaries provide planning and self-observation tools

rather than objective measures of research quality, and their value depends on the completeness and relevance of the recorded history. The present beta release is an initial open-source implementation developed by a single researcher and may therefore contain bugs, incomplete modules, setup complexity, and dependencies on evolving third-party plugins or services, such as Ollama, that require additional configuration. In addition, automated scheduling currently relies on macOS `launchd`, real-time multi-user editing is not supported, and new vaults initially lack sufficient history for data-informed forecasts. Nevertheless, HOB demonstrates that a comprehensive Research Management System can be constructed on a plain-text, local-first foundation by combining the capabilities of Obsidian with automation, Bayesian statistics, KPI collection, and deployable AI. Future development should address bugs, streamline software initialization, extend support to Windows and Linux, simplify installation, improve collaboration, and validate the framework across disciplines. Because HOB-authored source code, templates, and configuration are openly available under AGPL-3.0, covered redistributed or network-deployed modifications remain subject to the license's corresponding-source obligations. A community of researchers and developers could therefore progressively address these limitations and evolve HOB beyond the requirements and perspective of its original author.

Supplementary Information

The HOB repository, including all source code, templates, and configuration, is available at `https://github.com/Bendlexane/HOB` [27]. The release documented here is archived on Zenodo under the concept DOI `https://doi.org/10.5281/zenodo.22059429`, which resolves to the most recent version; version 1.0 is archived separately as `https://doi.org/10.5281/zenodo.22059430`. HOB-authored material is released under AGPL-3.0, while bundled third-party components remain governed by their respective licenses. The vault includes a `GET STARTED.md` guide for first-time setup.

License

This manuscript is distributed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). The software it describes is distributed separately under the GNU Affero General Public License version 3 or later.

Trademark Notice

Obsidian is a trademark of Dynalist Inc. HOB is not affiliated with, endorsed by, or sponsored by Dynalist Inc. **HOB is a structured, automated vault built on Obsidian; it is not a distribution of the Obsidian application.**

Acknowledgements

The author thanks Andrej Karpathy for proposing the LLM Wiki pattern, Dominique Leca for developing its upstream Obsidian implementation, and the Obsidian community and developers of the other integrated and companion plugins—including Dataview, Templater, QuickAdd, Full Calendar, Make.md, home-tab, and Sky Background—whose work makes HOB possible.

Author Contributions

Manuel Tiburtini — Conceptualization, Prototyping, Software, Writing — Original Draft.

Declaration of Generative AI Use

Generative AI tools were used to assist with software development and with the drafting and language revision of this manuscript. All AI-generated code and text were reviewed, tested where applicable, corrected, and approved by the author under active supervision.

Conflict of Interest

The author declares no conflict of interest.

Funding

This research received no external funding.

Data Availability

HOB is fully open-source under AGPL-3.0. The vault structure, all scripts, and the Obsidian configuration are publicly available in the repository cited above and permanently archived on Zenodo (`https://doi.org/10.5281/zenodo.22059429`). The KPI SQLite database and posterior estimates are user-specific and stored locally.

Software Disclaimer

HOB is provided "as is," without warranties of any kind, whether express or implied, including warranties of merchantability, fitness for a particular purpose, accuracy, reliability, and non-infringement. To the extent permitted by applicable law, the author and contributors shall not be liable for any direct, indirect, incidental, special, consequential, or other loss or damage arising from the installation, configuration, use, inability to use, or malfunction of the software. Users remain responsible for validating outputs, maintaining appropriate backups, securing credentials and research data, and complying with applicable ethical, institutional, and legal requirements. This statement summarizes the warranty disclaimer and limitation of liability contained in AGPL-3.0; if any inconsistency arises, the full license text distributed with the software governs.

References

[1] Philip Atkinson. *Folk Tales of North East England*. CreateSpace Independent Publishing Platform, 2017.

[2] Center for Open Science. OSF getting started FAQs. Official documentation, 2026. Accessed 21 August 2026. URL: `https://help.osf.io/article/546-getting-started-faq-s`.

[3] Center for Open Science. OSF Projects transition FAQs. Official documentation, 2026. Accessed 21 August 2026. URL: `https://help.osf.io/article/725-faqs`.

[4] Corporation for Digital Scholarship. Zotero: Your personal research assistant. Project website, 2026. Accessed 21 August 2026. URL: `https://www.zotero.org/`.

[5] Ulrich Dirnagl and Ingo Przesdzing. A pocket guide to electronic laboratory notebooks in the academic life sciences. *F1000Research*, 5:2, 2016. `doi:10.12688/f1000research.7628.1`.

[6] eLabFTW. eLabFTW official documentation. Project documentation, 2026. Accessed 21 August 2026. URL: https://doc.elabftw.net/.

[7] eLabFTW. Import and export documentation. Project documentation, 2026. Accessed 21 August 2026. URL: https://doc.elabftw.net/docs/usage/import-export/.

[8] Erin D. Foster and Ariel Deardorff. Open science framework (OSF). *Journal of the Medical Library Association*, 105(2):203–206, 2017. doi:10.5195/jmla.2017.88.

[9] Andrej Karpathy. LLM Wiki: A pattern for building personal knowledge bases using LLMs. GitHub Gist, 2026. Created 4 April 2026; accessed 22 August 2026. URL: https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f.

[10] Byung-Cheol Kim and Kenneth F. Reinschmidt. Probabilistic forecasting of project duration using bayesian inference and the beta distribution. *Journal of Construction Engineering and Management*, 135(3):178–186, 2009. doi:10.1061/(ASCE)0733-9364(2009)135:3(178).

[11] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pages 154–178. Association for Computing Machinery, 2019. doi:10.1145/3359591.3359737.

[12] Bianca Kramer and Jeroen Bosman. Innovations in scholarly communication—global survey on research tool usage. *F1000Research*, 5:692, 2016. doi:10.12688/f1000research.8414.1.

[13] LabArchives. ELN for research: Introduction and subscription plans. Official documentation, 2026. Accessed 21 August 2026. URL: https://www.labarchives.com/pricing.

[14] LabArchives. Proofig AI and LabArchives integration. Product documentation, 2026. Accessed 21 August 2026. URL: https://www.labarchives.com/integrations/proofig-ai-x-labarchives.

[15] LabArchives. Publishing data by generating a DOI. Official documentation, 2026. Accessed 21 August 2026. URL: https://help.labarchives.com/hc/en-us/articles/13593568250132-Publishing-Data-by-Generating-a-DOI.

[16] Dominique Leca. LM Wiki: An obsidian implementation of the LLM Wiki pattern. Software repository, 2026. MIT License; accessed 22 August 2026. URL: https://github.com/domleca/llm-wiki.

[17] Obsidian. Community plugins. Official documentation, 2026. Accessed 21 August 2026. URL: https://help.obsidian.md/extending-obsidian/community-plugins.

[18] Obsidian. CSS snippets. Official documentation, 2026. Accessed 21 August 2026. URL: https://help.obsidian.md/appearance/css-snippets.

[19] Obsidian. How obsidian stores data. Official documentation, 2026. Accessed 21 August 2026. URL: https://help.obsidian.md/files-and-folders/how-obsidian-stores-data.

[20] Obsidian. Obsidian: Sharpen your thinking. Project website, 2026. Accessed 21 August 2026. URL: https://obsidian.md/.

[21] Research Space. RSpace Community, Team and Enterprise Editions. Official documentation, 2026. Accessed 21 August 2026. URL: https://documentation.researchspace.com/l/en/article/8fzrfsbc5e-rspace-editions.

[22] Research Space. RSpace: Open-source research data management. Project website, 2026. Accessed 21 August 2026. URL: https://www.researchspace.com/.

[23] SciNote LLC. Export all your data in a readable format from SciNote. Product documentation, 2021. Accessed 21 August 2026. URL: https://www.scinote.net/blog/export-all-your-data/.

[24] SciNote LLC. Open-source electronic lab notebook. Product documentation, 2026. Accessed 21 August 2026. URL: https://www.scinote.net/open-source-code/.

[25] SciNote LLC. SciNote AI: Automate SOPs and protocols. Product documentation, 2026. Accessed 21 August 2026. URL: https://www.scinote.net/product/ai-and-automations/.

[26] SciNote LLC. Terms of service. Official documentation, 2026. Accessed 21 August 2026. URL: https://www.scinote.net/legal/terms-of-service/.

[27] Manuel Tiburtini. HOB: A free, open-source obsidian-based research management system for scientific workflows, 2026. Concept DOI, resolves to the latest release. URL: https://github.com/Bendlexane/HOB, doi:10.5281/zenodo.22059429.

[28] Eva A. M. van Dis, Johan Bollen, Willem Zuidema, Robert van Rooij, and Claudi L. Bockting. ChatGPT: Five priorities for research. *Nature*, 614:224–226, 2023. doi:10.1038/d41586-023-00288-7.